JAVASCRIPT · FUNDAMENTOS · CONTROL DE FLUJO

Fundamentos, decisiones y bucles

Una guía de apoyo para comprender los conceptos necesarios antes de resolver los ejercicios de los grupos 1 y 2. Pensada como teoría, referencia rápida y ayuda directa para programar.

console.log() · variables · prompt() · if / else · switch · for / while · break / continue · validación

01 Mostrar información

Al empezar con JavaScript necesitamos ver qué está haciendo nuestro programa. Las dos formas más sencillas son la consola y las alertas.

console.log()

Muestra información en la consola del navegador. Es la opción más útil para programar y depurar.

alert()

Muestra una ventana emergente al usuario. Úsala solo cuando el ejercicio lo pida.

JavaScript

```
console.log("Hola, JS");
alert("Bienvenidos");
```

Idea mental útil

Piensa en `console.log()` como una forma de “mirar dentro” del programa para comprobar valores y resultados.

02 Variables y tipos básicos

Una variable guarda un dato para utilizarlo más tarde. En estos ejercicios trabajarás sobre todo con textos, números y valores booleanos.

Tipo	Ejemplo	Uso
string	`"Ane"`	Texto
number	`19`	Números enteros o decimales
boolean	`true`	Verdadero / falso

JavaScript

```
const nombre = "Ana";
let edad = 19;

console.log(nombre);
console.log(edad);
```

const

Úsalo cuando no vas a reasignar la variable.

let

Úsalo cuando el valor vaya a cambiar.

03 Pedir datos al usuario

`prompt()` permite pedir un dato. El detalle importante es que devuelve texto, incluso cuando el usuario escribe un número.

JavaScript

```
const nombre = prompt("¿Cómo te llamas?");
console.log("Hola " + nombre);
```

Error típico

Si escribes `10` en un `prompt()`, JavaScript recibe `"10"`, no el número 10.

prompt() → **texto** → **Number(...)** → **número**

04 Operadores aritméticos

Los operadores permiten realizar cálculos básicos.

Operador	Significado	Ejemplo
+	Suma	`a + b`
-	Resta	`a - b`
*	Multiplicación	`a * b`
/	División	`a / b`
%	Resto de una división	`a % b`

JavaScript

```
const a = Number(prompt("A:"));
const b = Number(prompt("B:"));

console.log("Suma:", a + b);
console.log("Resta:", a - b);
console.log("Producto:", a * b);
console.log("División:", a / b);
```

05 Template literals

Los template literals permiten insertar variables dentro de un texto utilizando comillas invertidas.

JavaScript

```
const producto = "Teclado";
const precio = 19.9;

console.log(`El ${producto} cuesta ${precio.toFixed(2)}€`);
```

Idea mental útil

`${...}` significa: “mete aquí el valor de esta expresión”.

Muy útil en los ejercicios

`toFixed(2)` permite mostrar un número con dos decimales.

06 Conversión y NaN

Para convertir texto a número podemos usar `Number()`. Si el texto no representa un número válido, el resultado será `NaN`.

JavaScript

```
const entrada = "123abc";
const numero = Number(entrada);

if (Number.isNaN(numero)) {
  console.log("Entrada no numérica");
}
```

No confundas

`NaN` significa “Not a Number”. Para comprobarlo de forma clara utiliza `Number.isNaN(valor)`.

07 Comparaciones y operadores lógicos

Las condiciones producen un valor `true` o `false`. Después podemos usar ese resultado dentro de un `if`.

Operador	Significado
`===`	Igualdad estricta
`!==`	Distinto
`>` `<` `>=` `<=`	Comparaciones numéricas
`&&`	Y: ambas condiciones deben cumplirse
`||`	O: basta con que se cumpla una
`!`	Niega una condición

JavaScript

```
const nota = 8;

if (nota >= 5) {
  console.log("Aprobado");
}

if (nota >= 9) {
  console.log("Excelente");
}
```

08 Asignación e incremento

Cuando una variable cambia, es habitual actualizarla a partir de su valor anterior.

Forma	Equivale a
`x += 1`	`x = x + 1`
`x -= 1`	`x = x - 1`
`x++`	Incrementar 1
`x--`	Restar 1

JavaScript

```
let contador = 0;

contador += 1;
contador += 1;
contador += 1;

console.log(contador); // 3
```

09

Math y números aleatorios

`Math.random()` genera un número decimal entre 0 y 1. Para obtener enteros necesitamos combinarlo con otras operaciones.

Math.random() → **× 6** → **Math.floor()** → **+ 1**

JavaScript

```
const dado = Math.floor(Math.random() * 6) + 1;
console.log(dado);
```

Resultado

El valor final estará entre 1 y 6.

10 Strings útiles

Los textos tienen métodos que permiten limpiarlos y normalizarlos.

trim()

Elimina espacios al principio y al final.

toLowerCase()

Convierte el texto a minúsculas.

JavaScript

```
const email = " Usuario@Correo.com ";

const limpio = email.trim().toLowerCase();
console.log(limpio);
```

Por qué es útil

Permite comparar entradas del usuario sin que las mayúsculas o los espacios provoquen errores innecesarios.

11 Código legible y comentarios

Un programa correcto también debe ser fácil de entender. Usa nombres claros, sangrado y comentarios solo cuando aporten información útil.

JavaScript

```
// Cálculo del área de un rectángulo
const base = 5;
const altura = 3;

const area = base * altura;

console.log(`Área: ${area} cm²`);
```

Evita

Nombres como `x`, `a1` o `cosa` cuando puedes usar `precio`, `altura` o `contador`.

12 if / else: tomar una decisión

Un `if` ejecuta un bloque si una condición es verdadera. El `else` cubre el caso contrario.

JavaScript

```
const edad = Number(prompt("Edad:"));

if (edad >= 18) {
  console.log("Mayor de edad");
} else {
  console.log("Menor de edad");
}
```

condición → **true: if** / **false: else**

13 else if: más de dos posibilidades

Cuando existen varias opciones mutuamente excluyentes podemos encadenar condiciones con `else if`.

JavaScript

```
const n = Number(prompt("Número:"));

if (n < 0) {
  console.log("Negativo");
} else if (n === 0) {
  console.log("Cero");
} else {
  console.log("Positivo");
}
```

Orden importante

JavaScript comprueba las condiciones de arriba abajo y se queda con la primera que sea verdadera.

14 switch: elegir entre opciones concretas

`switch` es útil cuando comparas una variable con varios valores concretos, por ejemplo las opciones de un menú.

JavaScript

```
const opcion = Number(prompt("1 Sumar / 2 Restar"));

switch (opcion) {
  case 1:
    console.log("Has elegido sumar");
    break;

  case 2:
    console.log("Has elegido restar");
    break;

  default:
    console.log("Opción inválida");
}
```

Error típico

Olvidar `break` hace que el programa siga entrando en los siguientes casos.

15 for: repetir un número conocido de veces

El bucle `for` suele utilizarse cuando sabemos cuántas repeticiones queremos realizar.

JavaScript

```
for (let i = 1; i <= 10; i++) {
  console.log(i);
}
```

Parte	Significado
`let i = 1`	Valor inicial
`i <= 10`	Condición para continuar
`i++`	Cambio después de cada vuelta

16 while: repetir mientras se cumpla una condición

`while` es adecuado cuando no sabemos cuántas veces será necesario repetir.

JavaScript

```
const secreto = 3;
let intento;

while (intento !== secreto) {
  intento = Number(prompt("Adivina el número"));
}

console.log("¡Acertaste!");
```

Cuidado con los bucles infinitos

Dentro del bucle debe ocurrir algo que permita que la condición llegue a ser falsa.

17 for...of también funciona con strings

Un texto puede recorrerse carácter a carácter usando `for...of`.

JavaScript

```
const texto = "Programación".toLowerCase();

for (const caracter of texto) {
  console.log(caracter);
}
```

Ejemplo · contar vocales

```
let contador = 0;

for (const caracter of texto) {
  if ("aeiou".includes(caracter)) {
    contador++;
  }
}

console.log(contador);
```

18 break y continue

Ambas instrucciones cambian el comportamiento normal de un bucle.

continue

Salta el resto de la vuelta actual y pasa a la siguiente.

break

Termina el bucle por completo.

JavaScript

```
for (let i = 1; i <= 20; i++) {
  if (i % 3 === 0) {
    continue;
  }

  if (i === 17) {
    break;
  }

  console.log(i);
}
```

19 do...while: ejecutar al menos una vez

La diferencia con `while` es que el bloque se ejecuta primero y la condición se comprueba después.

JavaScript

```
let opcion;

do {
  opcion = prompt("1 Info / 2 Ayuda / 0 Salir");
} while (opcion !== "0");
```

Cuándo encaja bien

Menús y validaciones en las que necesitamos pedir el dato al menos una vez.

20 Validar una entrada

Validar significa comprobar que el dato recibido cumple lo que necesitamos antes de seguir.

JavaScript

```
let numero;

do {
  numero = Number(prompt("Introduce un número válido:"));
} while (Number.isNaN(numero));

console.log("OK:", numero);
```

pedir dato → **convertir** → **comprobar** → **repetir si falla**

21 Contadores y mini-quiz

Muchos ejercicios necesitan contar cuántas veces ocurre algo. Para eso se crea una variable que empieza normalmente en 0 y se incrementa cuando se cumple una condición.

JavaScript

```
let aciertos = 0;

const respuesta = prompt("2 + 2 = ?");

if (respuesta === "4") {
  aciertos++;
}

console.log("Aciertos:", aciertos);
```

Patrón que debes reconocer

Crear contador → comprobar condición → incrementar → mostrar resultado final.

22 Método para resolver los ejercicios

Antes de escribir código, divide el problema en pasos pequeños.

1. **Datos.** ¿Qué información tengo que pedir o guardar?
2. **Tipo.** ¿Es texto o número? ¿Tengo que usar `Number()`?
3. **Objetivo.** ¿Qué resultado final debe aparecer?
4. **Decisión.** ¿Necesito `if`, `else if` o `switch`?
5. **Repetición.** ¿Necesito un bucle? ¿Sé cuántas veces o depende de una condición?
6. **Estado.** ¿Necesito un contador, acumulador o una variable que cambie?
7. **Prueba.** Comprueba varios casos: normal, límite y entrada incorrecta.

Objetivo final

Que puedas mirar el enunciado y reconocer qué herramientas básicas necesitas antes de empezar a programar.

23 Chuleta rápida

Referencia mínima para tener a mano mientras programas.

Quiero...	Uso...
Mostrar información	`console.log(...)`
Pedir un dato	`prompt(...)`
Convertir a número	`Number(...)`
Comprobar número inválido	`Number.isNaN(...)`
Insertar variables en texto	`` `Hola ${nombre}` ``
Dos caminos	`if / else`
Varias condiciones	`if / else if / else`
Elegir opción concreta	`switch`
Repetir N veces	`for`
Repetir mientras...	`while`
Ejecutar al menos una vez	`do...while`
Recorrer caracteres	`for...of`
Saltar una vuelta	`continue`
Terminar un bucle	`break`
Incrementar contador	`contador++`